

DSA DAILY — FROM BASICS TO ADVANCED

# DAY 14 — PREFIX SUM

Learn → Understand → Visualize → Code → Practice → Revise

**INTERACTIVE EDITION**

[Clickable contents](#) • [Previous/Next navigation](#) • [External practice links](#)

# INTERACTIVE CONTENTS

Click any section below to jump directly to that page.

- [01 Cumulative information, made reusable](#)
- [02 Understand every value](#)
- [03 Repeated range sums are expensive](#)
- [04 Build once, reuse many times](#)
- [05 The interview formula to remember](#)
- [06 Prefix Sum cancels the unwanted part](#)
- [07 Clean, interview-ready code](#)
- [08 What each important line is doing](#)
- [09 Where Prefix Sum really shines](#)
- [10 A major interview pattern](#)
- [11 Example: `nums = \[1, 2, 3\]`, `k = 3`](#)
- [12 Prefix Sum does not require positive values](#)
- [13 Turn a condition into 0 / 1](#)
- [14 Extend the idea from arrays to matrices](#)
- [15 Choose the pattern from the problem](#)
- [16 Trade memory for faster queries](#)
- [17 Small indexing mistakes can break the solution](#)
- [18 Cumulative information appears everywhere](#)
- [19 Build the pattern through deliberate practice](#)
- [20 Day 14 → mastered](#)

[Open GitHub Repository →](#)

# 01 WHAT IS PREFIX SUM? | Cumulative information, made reusable

A Prefix Sum stores cumulative totals so we do not repeatedly add the same elements.

Formula

```
prefix[i + 1] = prefix[i] + arr[i]
```

Example

Array: [2, 4, 1, 3, 5]

Prefix: [0, 2, 6, 7, 10, 15]

Why the leading 0?

It represents the sum of zero elements and makes every range formula consistent.

## 02 READ THE PREFIX ARRAY | Understand every value

```
arr[0] = 2 → prefix[1] = 2  
arr[1] = 4 → prefix[2] = 6  
arr[2] = 1 → prefix[3] = 7  
arr[3] = 3 → prefix[4] = 10  
arr[4] = 5 → prefix[5] = 15
```

Think of `prefix[x]` as:

“the sum of the first `x` elements.”

Original array index `i` maps to prefix index `i + 1`.

## 03 WHY DO WE NEED IT? | Repeated range sums are expensive

### Brute Force

For every query (L, R), loop from L to R.

- One query:  $O(n)$  in the worst case
- Q queries:  $O(n \times Q)$

### Prefix Sum

- Preprocessing:  $O(n)$
- Each range query:  $O(1)$
- Q queries:  $O(n + Q)$

The key idea:

Do the repeated work once, then reuse the result.

## 04 HOW PREFIX SUM WORKS | Build once, reuse many times

arr = [3, 1, 4, 2, 5]

```
prefix[0] = 0
prefix[1] = 0 + 3 = 3
prefix[2] = 3 + 1 = 4
prefix[3] = 4 + 4 = 8
prefix[4] = 8 + 2 = 10
prefix[5] = 10 + 5 = 15
```

Index 0 1 2 3 4 5

Array 3 1 4 2 5 —

Prefix 0 3 4 8 10 15

## 05 THE RANGE-SUM FORMULA | The interview formula to remember

```
sum(L, R) = prefix[R + 1] - prefix[L]
```

Why it works:

```
prefix[R + 1] contains everything from the beginning through R.
```

```
prefix[L] contains everything before L.
```

Subtracting removes the unwanted left part.

Example

```
arr = [2, 4, 1, 3, 5, 2]
```

```
prefix = [0, 2, 6, 7, 10, 15, 17]
```

```
L = 2, R = 5
```

```
sum(2, 5) = prefix[6] - prefix[2]
```

```
= 17 - 6
```

```
= 11
```

Selected subarray: [1, 3, 5, 2]



## 06 RANGE QUERY — VISUAL INTUITION | Prefix Sum cancels the unwanted part

PREFIX

[ 0 | 2 | 6 | 7 | 10 | 15 | 17 ]

↑ ↑

prefix[L] prefix[R+1]

Range L=2 to R=5

$17 - 6 = 11$

The subtraction removes:

$2 + 4 = 6$

What remains:

$1 + 3 + 5 + 2 = 11$

## 07 C++ IMPLEMENTATION | Clean, interview-ready code

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    vector<int> arr = {2, 4, 1, 3, 5, 2};
    int n = arr.size();
    vector<long long> prefix(n + 1, 0);
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + arr[i];
    }
    int L = 2, R = 5;
    long long rangeSum = prefix[R + 1] - prefix[L];
    cout << rangeSum << endl;
    return 0;
}
```

## 08 CODE WALKTHROUGH | What each important line is doing

```
vector<long long> prefix(n + 1, 0);
```

→ Extra slot for `prefix[0] = 0`.

```
prefix[i + 1] = prefix[i] + arr[i];
```

→ Add the current value to the cumulative total.

```
prefix[R + 1] - prefix[L];
```

→ Convert an inclusive `[L, R]` query into `O(1)`.

Why long long?

A cumulative sum can become much larger than an individual int value.

Using long long gives a wider integer range for the total.

## 09 MULTIPLE RANGE QUERIES | Where Prefix Sum really shines

Array: [2, 4, 1, 3, 5, 2]

Prefix: [0, 2, 6, 7, 10, 15, 17]

Query (0,3):

```
prefix[4] - prefix[0] = 10
```

Query (2,5):

```
prefix[6] - prefix[2] = 11
```

Query (1,4):

```
prefix[5] - prefix[1] = 13
```

After  $O(n)$  preprocessing, every query is  $O(1)$ .

Total:  $O(n + Q)$ .

# 10 PREFIX SUM + HASH MAP | A major interview pattern

Suppose:

$$\text{prefix}[j] - \text{prefix}[i] = k$$

Rearrange:

$$\text{prefix}[i] = \text{prefix}[j] - k$$

So while scanning the array:

1. Maintain the current prefix sum.
2. Look for  $\text{currentPrefix} - k$  in a hash map.
3. If it exists, matching subarrays end here.
4. Store the current prefix sum frequency.

Important initialization:

$$\text{freq}[0] = 1$$

This represents the empty prefix and allows a valid subarray to start at index 0.

## 11 SUBARRAY SUM EQUALS K | Example: nums = [1, 2, 3], k = 3

Start: freq = {0: 1}, prefix = 0, answer = 0

Read 1:

prefix = 1, need = -2 → not found

store 1

Read 2:

prefix = 3, need = 0 → found once

valid subarray: [1,2]

answer = 1

store 3

Read 3:

prefix = 6, need = 3 → found once

valid subarray: [3]

answer = 2

store 6

Final answer = 2

Valid subarrays: [1,2] and [3]

## 12 NEGATIVE NUMBERS | Prefix Sum does not require positive values

Array: [5, -2, 4, -1]

Prefix:

[0, 5, 3, 7, 6]

Example range [1,3]:

```
sum = prefix[4] - prefix[1]
= 6 - 5
= 1
```

Selected values:

[-2, 4, -1] = 1

Key point:

Prefix Sum works with negative numbers too.

It simply records cumulative totals.

# 13 PREFIX SUM FOR COUNTING | Turn a condition into 0 / 1

Binary array:

[1, 0, 1, 1, 0, 1]

Prefix:

[0, 1, 1, 2, 3, 3, 4]

To count 1s from L to R:

```
count = prefix[R + 1] - prefix[L]
```

Example [1,4]:

```
prefix[5] - prefix[1]
```

= 3 - 1

= 2

General pattern:

condition true  $\rightarrow$  1

condition false  $\rightarrow$  0

Then apply Prefix Sum.



## 14 2D PREFIX SUM | Extend the idea from arrays to matrices

For a matrix, `prefix[r][c]` stores cumulative information for a rectangle.

Rectangle sum concept:

top + left – overlap + current

Using 1-based padded indexing:

$\text{sum} = P[r2+1][c2+1]$

–  $P[r1][c2+1]$

–  $P[r2+1][c1]$

+  $P[r1][c1]$

Why + overlap?

The top-left region was subtracted twice, so add it back once.

Useful for:

- image processing
- grid queries
- matrix / rectangle-sum problems

# 15 PREFIX SUM vs OTHER TECHNIQUES | Choose the pattern from the problem

## PREFIX SUM

Best for: range sums / cumulative information

Typical clue: many range queries

## SLIDING WINDOW

Best for: contiguous windows with constraints

Typical clue: maintain a moving window

## TWO POINTERS

Best for: sorted arrays / moving boundaries

Typical clue: two indices expand, shrink, or meet

Interview habit:

Do not choose a technique by name—choose it from the structure of the problem.

# 16 TIME & SPACE COMPLEXITY | Trade memory for faster queries

Prefix construction

$O(n)$

One range query

$O(1)$

Q range queries

$O(n + Q)$

Space

$O(n)$

Trade-off:

More preprocessing + memory

→ much faster repeated queries

This is the central advantage of Prefix Sum:  
turn repeated work into reusable information.

# 17 COMMON MISTAKES | Small indexing mistakes can break the solution

## ■ Off-by-one errors

Use the inclusive range formula carefully.

## ■ Incorrect $R + 1$

For  $[L, R]$ , use  $\text{prefix}[R + 1] - \text{prefix}[L]$ .

## ■ Forgetting $\text{prefix}[0] = 0$

The padded zero makes boundary ranges easy.

## ■ int overflow

Use long long when cumulative totals may be large.

## ■ Confusing Prefix Sum with Sliding Window

They solve different kinds of problems.

## ■ Forgetting $\text{freq}[0] = 1$

Critical in Prefix Sum + Hash Map patterns.

## ■ Misunderstanding negatives

Negative values are valid in Prefix Sum.

# 18 REAL-WORLD APPLICATIONS | Cumulative information appears everywhere

Daily sales → sales between two dates

Bank transactions → totals over a period

Website traffic → visits over a time range

Sensor data → measurements across intervals

Gaming scores → score totals over rounds

Analytics dashboards → fast range metrics

Financial reports → cumulative values by period

Image processing → fast rectangle / grid calculations

The common idea:

precompute cumulative information, then reuse it.

# 19 PRACTICE + INTERVIEW TIP | Build the pattern through deliberate practice

Recommended LeetCode order

303. Range Sum Query - Immutable

→ Learn basic range-query Prefix Sum

Difficulty: Beginner

724. Find Pivot Index

→ Practice total sum + left/right cumulative sums

Difficulty: Intermediate

560. Subarray Sum Equals K

→ Learn Prefix Sum + Hash Map

Difficulty: Interview Level

303 → 724 → 560

## INTERVIEW CALLOUT

“Whenever you see multiple range-sum queries, think Prefix Sum.”

A strong explanation:

“I’ll preprocess cumulative sums in  $O(n)$ . Then each range query is  $O(1)$  using  $\text{prefix}[R+1] - \text{prefix}[L]$ , giving  $O(n+Q)$  overall.”

# 20 CHEAT SHEET + TOMORROW | Day 14 → mastered

## FORMULA

```
prefix[i+1] = prefix[i] + arr[i]
```

## RANGE SUM

```
sum(L,R) = prefix[R+1] - prefix[L]
```

BUILD  $O(n)$

QUERY  $O(1)$

Q QUERIES  $O(n + Q)$

SPACE  $O(n)$

IMPORTANT PATTERN

**Prefix Sum + Hash Map**

INTERVIEW TRIGGER

Many range queries → Prefix Sum

## TOMORROW

Day 15 — Difference Array / Range Updates

A related idea for making repeated range updates efficient.

DSA Daily — From Basics to Advanced

■ [GitHub Repository](#)